claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_1c5f1f5c-9ba\agent-a3cc133b9a99046a7.jsonl`
- SHA-256: `b607dc95a23257cb942d92b72f3e8b85e47ffc5814907efc3d1c431d08cd493d`
- Source modified: `2026-06-20T08:05:13+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_1c5f1f5c-9ba`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`

Transcript

1. user (2026-06-20T08:03:26.399Z)

Design the KhelSutra "interactive per-rally selection" feature using this lens:
TIMELINE-FIRST: the dominant surface is a scrubbable match timeline with rally markers; the user toggles rallies on/off directly on the timeline and previews inline. Selection state drives a live reel preview (total clips + duration).

The feature (owner's words): the engine detects rallies -> present them in a UI so the user PICKS which rallies go into the highlight reel ? dynamic per-video, NOT a fixed threshold. It lives in the alpha SPA (khelsutra repo, web/) over the rally index the engine produces. It pairs with the conversational "ask for a reel" builder.

GROUND EVERY DECISION in this real research about the actual engine (data contract, API, UI plans, doc convention). Do NOT propose consuming fields that don't exist today (shot type, player, score are roadmap-only unless the research says otherwise). Be honest about what needs new engine API vs what exists:


```
{
  "contract": {
    "tables": [
      {
        "name": "videos",
        "purpose": "One row per ingested video file. Parent of all segments (FK cascade). Status
is the ingest verdict (processed/failed). user_id (v4) is NULL on local installs.",
        "columns": [
          "id TEXT PRIMARY KEY (= MD5 of the whole file, compute_video_id)",
          "filepath TEXT NOT NULL",
          "processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP",
          "status TEXT NOT NULL (e.g. 'processed' | 'failed')",
          "validation_results TEXT (JSON list of ValidationResult dicts)",
          "user_id TEXT (NULLABLE, added v4; NULL = local install)"
        ]
      },
      {
        "name": "play_segments",
        "purpose": "CONFIRMED rallies ? the table a per-rally selection UI consumes today.
/api/compile and /api/query read here. One row per accepted rally.",
        "columns": [
          "id INTEGER PK AUTOINCREMENT (the segment_id the UI selects/stitches)",
          "video_id TEXT NOT NULL (FK videos.id ON DELETE CASCADE)",
          "start_time REAL NOT NULL (seconds)",
          "end_time REAL NOT NULL (seconds)",
          "duration REAL NOT NULL (= end-start, computed at insert)",
          "server TEXT (NULLABLE; only the motion segmenter sets it, and it is
FABRICATED/random)",
          "receiver TEXT (NULLABLE; same ? motion-only, fabricated)",
```

"metadata TEXT (extensible JSON; gemini writes {shot_count, shot_count_confidence, detector}; motion writes {shot_count, intensity, avg_speed_kmh} all random)"
        ]
      },
      {
        "name": "candidate_segments",
        "purpose": "RAW pre-confirmation detections (detector-agnostic), persisted by the hybrid segmenters only. Holds the human-correction-loop columns (human_label/notes/confirmed_segment_id) that a review UI would write ? but NO endpoint writes them yet.",
        "columns": [
          "id INTEGER PK AUTOINCREMENT",
          "video_id TEXT NOT NULL (FK videos.id ON DELETE CASCADE)",
          "detector TEXT NOT NULL (e.g. wasb_hybrid/yolo_hybrid/fusion key)",
          "chunk_index INTEGER (NULLABLE)",
          "start_time REAL NOT NULL",
          "end_time REAL NOT NULL",
          "duration REAL NOT NULL",
          "confidence REAL (NULLABLE)",
          "stats TEXT (detector-specific JSON metrics: peak/mean velocity, active_frame_count, net_crossings, etc.)",
          "detection_params TEXT (JSON of the detector knobs used)",
          "confirmed_segment_id INTEGER (NULLABLE FK play_segments.id ON DELETE SET NULL ? link to the confirmed rally)",
          "human_label TEXT (NULLABLE ? correction-loop label, UNUSED by any code path today)",
          "notes TEXT (NULLABLE ? correction-loop notes, UNUSED today)",
          "created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP",
          "user_id TEXT (NULLABLE, added v4)"
        ]
      },
      {
        "name": "events",
        "purpose": "Sub-rally events keyed to a play_segment. ONLY the motion segmenter writes these, and every value is RANDOM/fabricated (not ground truth). Gemini/hybrid paths write NO events.",
        "columns": [
          "id INTEGER PK AUTOINCREMENT",
          "segment_id INTEGER NOT NULL (FK play_segments.id ON DELETE CASCADE)",
          "timestamp REAL NOT NULL",
          "type TEXT NOT NULL ('serve' | 'smash' | 'foul' | 'winner')",
          "player TEXT (NULLABLE; from a fabricated player_pool)",
          "details TEXT (JSON; e.g. {speed_kmh}, {type})"
        ]
      },
      {
        "name": "jobs",
        "purpose": "Async job store (v2/v3). One row per /api/process submission. status = EXECUTION state; the pipeline verdict lives inside result JSON. Not rally data, but the UI polls GET /api/jobs/{id} for progress.",
        "columns": [
          "id TEXT PK (uuid hex)",
          "video_path TEXT NOT NULL",
          "video_id TEXT (NULLABLE)",
          "segmenter TEXT",
          "player_pool TEXT (JSON)",
          "max_frames INTEGER",
          "status TEXT NOT NULL DEFAULT 'queued' (queued|running|waiting|done|failed)",
          "progress TEXT",
          "error TEXT",
          "result TEXT (JSON full response dict incl. play_segments + report paths)",
          "created_at/started_at/finished_at TIMESTAMP",
          "policy TEXT (v3 JSON ExecutionPolicy)",
          "next_poll_at TIMESTAMP (v3)"
        ]
      },

```
      {
        "name": "user_models",
        "purpose": "Per-user model store (v5). Personalization groundwork ONLY ? nothing on the served path reads it. Not consumed by a rally-selection UI.",
        "columns": [
          "id, user_id, version, baseline_version, feature_schema_hash, fusion_config(JSON), classifier_json(JSON), promotion_evidence(JSON), n_videos_at_fit, is_active, created_at"
        ]
      }
    ],
    "rally_fields_today": [
      "play_segments.id ? the integer the UI selects and passes to /api/compile as segment_ids (backend/infrastructure/database.py:125-135, backend/main.py:102-105,386)",
      "start_time / end_time / duration (REAL seconds) ? the interval the reel is cut from; duration computed at insert as end-start (database.py:127-129,347)",
      "video_id ? links the rally to its source file; /api/compile resolves videos.filepath as the raw source to cut (main.py:344,393; database.py:418-425)",
      "metadata JSON.shot_count + shot_count_confidence ? gemini (default) persists these from Gemini's shuttle_exchange_count + confidence; falls back to max(3,int(dur/0.8)) when absent (gemini.py:470-485)",
      "metadata JSON.detector ? the model_name string (e.g. 'gemini-2.5-flash') (gemini.py:484)",
      "server / receiver ? present in schema and returned by query, but ONLY the 'motion' segmenter sets them and they are random.sample fabrications, NOT real player identity (motion.py:191,201; database.py:130-131)",
      "events (type/player/timestamp/details) ? serve/smash/foul/winner; ONLY motion writes them and ALL values are random.* fabrications, queryable via /api/query event_type filter (motion.py:177-252; database.py:451-453)",
      "candidate_segments.stats JSON (hybrid paths only) ? per-window peak/mean velocity, active_frame_count, net_crossings; raw detector metrics, not surfaced by /api/query (database.py:362-383)",
      "candidate_segments.confidence ? REAL, persisted by add_candidate_segment but currently passed as None by the trajectory hybrids (database.py:163,362-383)"
    ],
    "rally_fields_roadmap": [
      "shot_type (smash/clear/drop/drive/net) per shot ? NO detection or column exists; events.type only carries motion's fabricated serve/smash/foul/winner labels, never produced by gemini/hybrid paths (motion.py:228-239)",
      "player identity per rally/shot ? NO real detection. play_segments.server/receiver and events.player are random.sample picks from a hardcoded pool in the motion demo only; PersonDetector tracks anonymous chunk-local ByteTrack IDs, never named players (motion.py:25-26,191; person_detector.py make_tracker)",
      "match score / point score per rally ? ABSENT everywhere. No score/game/set column in any table; grep for score/game_index/set_index in backend finds only eval-F1 'score' and a detector_score_threshold config knob, never a match score (database.py whole file; config/models.py:126)",
      "game/set index (which game of the match a rally belongs to) ? NO column and no detection; rallies are a flat per-video list with no game/set grouping (database.py:122-135)",
      "confidence on confirmed rallies ? play_segments has no confidence column; only candidate_segments.confidence exists and the live trajectory hybrids pass None into it (database.py:122-135 vs 362-383)",
      "human correction state (accepted/rejected/nudged) ? columns EXIST (candidate_segments.human_label, notes, confirmed_segment_id) but are dead: no API endpoint or code path writes human_label/notes; UI_PROPOSAL B6 PATCH /api/segments/{id} is proposed, not built (database.py:166-168; docs/UI_PROPOSAL.md:59)"
    ],
    "reel_compose": {
      "entrypoint": "POST /api/compile -> backend/main.py:342-396 (compile_highlights) -> backend/pipeline/stitcher/compiler.py:303 VideoCompiler.compile_highlights. Batch/CLI variants: tools/generate_highlights.py and main.py --trim-start/--trim-end both call the same VideoCompiler.",
      "inputs": "CompileRequest = {video_id: str, segment_ids: List[int], output_filename: str} (main.py:102-105). The endpoint loads videos.filepath via db.get_video, fetches ALL play_segments via query_play_segments, filters to the requested segment_ids, applies the stitching.min_shot_count floor (segments without a known shot_count are exempt), then extracts
```

time_pairs = [(start_time,end_time)] (main.py:344-386).
VideoCompiler.compile_highlights(raw_video_path, segments:[(start,end)], output_filename) is the underlying contract ? it takes the RAW source path + interval pairs, NOT segment ids (compiler.py:303).",
      "outputs": "A single MP4 written to output_dir/<safe output_filename>; the endpoint returns {status:'success', filepath: out_path} (main.py:393-394). The browser then downloads it via GET /api/highlights/{video_id}/{filename} (FileResponse stream; main.py:307-328). There is NO summary/JSON sidecar emitted by compile ? the per-video observability report is a separate artifact emitted during processing, not at compile time.",
      "notes": "Pipeline: _merge_overlapping (sort + collapse overlapping/abutting/duplicate ranges so a rally never plays twice; drops end<=start) -> per-clip ffmpeg trim with begin_padding/end_padding (default 0.5/0.5 from StitchingConfig) RE-ENCODED libx264 superfast crf22 + aac, optional drawtext/logo watermark baked per-clip -> concat demuxer with -c copy (clips are codec-identical so concat stream-copies). Out-of-range/invalid segments are skipped with a logged summary; if ALL skip it raises. Selection is purely by segment_ids the caller supplies ? the backend has no 'select best N' logic."
    },
    "playback": {
      "proxy_available": "NO. There is no low-res proxy video generated or stored for UI scrubbing. The only downscaling is transient: gemini.py re-encodes/downscales upload chunks to indexing.ai_chunk_scale_width (default 1280) purely to fit the provider upload cap, and those chunk files are deleted immediately after analysis (gemini.py:189-235,531). No proxy/preview MP4 is persisted anywhere (grep for proxy/preview/low-res finds only docs + the chunk-scale knob).",
      "frame_access": "NO per-rally thumbnails or extracted frames exist for the UI. The only frame extraction is wasb_infer.py::extract_frames (backend/pipeline/detectors/wasb_infer.py:547-571) which decodes the whole video to index-named PNGs (00000000.png ...) inside the WSL detector cache as a transient input to the GPU shuttle detector; it is deleted on a successful run by the cache-hygiene logic (keep_frames default false) and is never keyed to rallies or served. There is no thumbnail column and no /api/frames endpoint.",
      "notes": "A per-rally clip UI today has two options, both requiring the RAW source file: (1) play the single compiled reel after /api/compile + download via GET /api/highlights/{video_id}/{filename}; (2) play the full source and seek to play_segments.start_time/end_time (the times are exact seconds). There is NO endpoint that streams an arbitrary [start,end] span or a single rally clip, and NO endpoint that serves the original source video ? only compiled reels under output_dir are downloadable (main.py:307-328). The current static UI (backend/api/static/app.js) has NO <video> element at all: it renders rally cards, lets the user check segment_ids, compiles, and offers a download link; it also reads data.rallies which is a known bug (the API returns play_segments), per UI_PROPOSAL B5 (app.js:344,378; UI_PROPOSAL.md:58). A real review UI would need to either serve the source file or add a per-span clip/proxy endpoint ? neither exists."
    },
    "file_refs": [
      "backend/infrastructure/database.py:109-148 (base tables: videos, play_segments, events)",
      "backend/infrastructure/database.py:122-135 (play_segments columns)",
      "backend/infrastructure/database.py:137-148 (events columns)",
      "backend/infrastructure/database.py:150-178 (v1 candidate_segments + columns)",
      "backend/infrastructure/database.py:166-168 (human_label, notes, confirmed_segment_id correction-loop columns)",
      "backend/infrastructure/database.py:180-205 (v2 jobs table)",
      "backend/infrastructure/database.py:207-217 (v3 jobs.policy/next_poll_at)",
      "backend/infrastructure/database.py:219-237 (v4 user_id on videos + candidate_segments)",
      "backend/infrastructure/database.py:239-285 (v5 user_models)",
      "backend/infrastructure/database.py:341-353 (add_play_segment: duration = end-start)",
      "backend/infrastructure/database.py:362-383 (add_candidate_segment, confidence/stats)",
      "backend/infrastructure/database.py:427-475 (query_play_segments + filters + events join ? the UI read path)",
      "backend/pipeline/segmenters/gemini.py:461-498 (_populate_db: metadata = shot_count, shot_count_confidence, detector; server/receiver = None)",
      "backend/sports/badminton.py:41-54 (Gemini output schema: start_time,end_time,shuttle_exchange_count,shot_count_confidence ? no shot_type/player/score)",
      "backend/pipeline/segmenters/motion.py:191-250 (fabricated server/receiver + random serve/smash/foul/winner events)",
      "backend/main.py:102-105 (CompileRequest = video_id, segment_ids, output_filename)",
      "backend/main.py:342-396 (/api/compile: select by segment_ids, min_shot_count floor,

time_pairs, VideoCompiler)",
      "backend/main.py:307-328 (/api/highlights/{video_id}/{filename} download ? only compiled reels, not source)",
      "backend/main.py:331-340 (/api/query returns play_segments)",
      "backend/pipeline/stitcher/compiler.py:80-105 (_merge_overlapping de-dupe)",
      "backend/pipeline/stitcher/compiler.py:303-413 (compile_highlights: per-clip re-encode -> concat -c copy -> single MP4)",
      "backend/pipeline/detectors/wasb_infer.py:547-571 (extract_frames: transient WSL PNG frames, deleted on success ? not UI thumbnails)",
      "backend/api/static/app.js:344,363-409 (UI: no video element, data.rallies bug, select segment_ids -> compile -> download)",
      "backend/config/models.py:126 (detector_score_threshold ? a config knob, NOT a per-rally match score)",
      "docs/UI_PROPOSAL.md:46,58-59 (A4 rally-review + B5 replace static UI + B6 PATCH /api/segments correction endpoints ? proposed, NOT implemented)"
    ]
  },
  "api": {
    "endpoints": [
      {
        "method": "GET",
        "path": "/",
        "purpose": "Serve the bundled static SPA shell (backend/api/static/index.html) or a placeholder message.",
        "response": "HTML (HTMLResponse). text/html string.",
        "auth": "None. backend/main.py:52"
      },
      {
        "method": "GET",
        "path": "/api/config",
        "purpose": "Return the live typed AppConfig the server started with (Pydantic model dump).",
        "response": "JSON: the full AppConfig object (indexing/stitching/output/security/etc). backend/main.py:121-123 returns global_config.",
        "auth": "None. Leaks full server config incl. security settings. backend/main.py:121"
      },
      {
        "method": "POST",
        "path": "/api/process",
        "purpose": "Submit a hash->validate->segment job for a server-local video_path. Fast-fails bad path / missing file / unknown segmenter, then enqueues async job.",
        "response": "202 JSON {job_id, status:'queued', poll:'/api/jobs/{job_id}'}. Body model ProcessRequest{video_path, player_pool?, max_frames?, segmenter?}. backend/main.py:240-271",
        "auth": "None. Path containment only IF security.allowed_video_root is set (validate_input_path), else any server-readable path. backend/main.py:240"
      },
      {
        "method": "GET",
        "path": "/api/jobs/{job_id}",
        "purpose": "Poll execution state of a submitted processing job.",
        "response": "JSON {job_id, status(queued|running|waiting|done|failed), progress, video_id, error, result(pipeline dict incl. play_segments), reports, created_at, started_at, finished_at}. 404 if unknown. backend/main.py:274-293",
        "auth": "None. Any caller can poll any job_id (no ownership check). backend/main.py:274"
      },
      {
        "method": "POST",
        "path": "/api/query",
        "purpose": "List/filter play (rally) segments for a video_id. This is the ONLY way to list rallies today.",
        "response": "JSON {play_segments: [{id, video_id, start_time, end_time, duration, server, receiver, metadata(dict), events:[{id,segment_id,timestamp,type,player,details}]}]}. Body QueryRequest{video_id, server?, receiver?, duration_min?, event_type?}. backend/main.py:331-340; shape from database.py:427-475",
        "auth": "None. Any caller can query any video_id's rallies. backend/main.py:331"

```
      },
      {
        "method": "POST",
        "path": "/api/compile",
        "purpose": "Stitch a CUSTOM set of selected rally segment_ids into one highlight reel
(applies stitching.min_shot_count floor + paddings + watermark).",
        "response": "JSON {status:'success', filepath: server-local out path}. 404 unknown
video; 400 no matching/all-below-floor; 500 stitch fail. Body CompileRequest{video_id,
segment_ids:[int], output_filename}. backend/main.py:342-396",
        "auth": "None. Returns a server-local filepath (not the download URL).
backend/main.py:342"
      },
      {
        "method": "GET",
        "path": "/api/highlights/{video_id}/{filename}",
        "purpose": "Stream/download a previously compiled highlight reel by bare filename.",
        "response": "Binary FileResponse (video/mp4 or video/quicktime). 400 bad/traversal name;
404 unknown video or no compiled file. backend/main.py:307-328",
        "auth": "None. Path-traversal guarded (safe_output_filename + resolve_under_root).
backend/main.py:307"
      },
      {
        "method": "POST",
        "path": "/api/upload/init",
        "purpose": "Begin a chunked browser upload (validates extension + total-size cap).",
        "response": "JSON {upload_id, max_part_mb, next_part:'/api/upload/{id}/part/0'}. 400 bad
ext/name; 413 too big. Body UploadInitRequest{filename, total_size_bytes?}.
backend/api/uploads.py:64-86",
        "auth": "None. Upload state is in-process, global (not per-user).
backend/api/uploads.py:64"
      },
      {
        "method": "PUT",
        "path": "/api/upload/{upload_id}/part/{index}",
        "purpose": "Append one strictly-sequential raw-bytes part to an in-flight upload
(enforces per-part + total caps).",
        "response": "JSON {upload_id, received_parts, received_bytes}. 404 unknown id; 409 out-
of-order; 413 overflow (aborts). backend/api/uploads.py:89-120",
        "auth": "None. backend/api/uploads.py:89"
      },
      {
        "method": "POST",
        "path": "/api/upload/{upload_id}/complete",
        "purpose": "Finalize a chunked upload: verify part count, move into upload_dir, compute
video_id.",
        "response": "JSON {path(abs server path), video_id(MD5), size_bytes, next:'POST
/api/process with this path'}. 404 unknown id; 409 part-count mismatch. Body
UploadCompleteRequest{total_parts}. backend/api/uploads.py:123-144",
        "auth": "None. backend/api/uploads.py:123"
      },
      {
        "method": "DELETE",
        "path": "/api/upload/{upload_id}",
        "purpose": "Abort an in-flight upload and delete its partial file.",
        "response": "JSON {status:'aborted', upload_id}. 404 unknown id.
backend/api/uploads.py:147-155",
        "auth": "None. backend/api/uploads.py:147"
      }
    ],
    "auth_model": "NONE today. Every endpoint is fully unauthenticated: there is no
Depends/Security dependency, no HTTPBearer, no API-key/token check, no DEV_USER_EMAIL read, and
no per-user scoping in any handler (verified across backend/**/*.py). CORS is wide-open:
allow_origins=['*'] with allow_credentials=False (backend/main.py:40-46). The DB schema is auth-
READY but unwired: videos.user_id and candidate_segments.user_id columns + partial indexes exist
(database.py:221-236, migration v4) and a per-user user_models table exists
```

```
(database.py:240-283, v5), but NO endpoint reads or writes user_id. INTENDED model (docs, not
built): alpha = Cloudflare Access in front of both hostnames; FastAPI trusts the 'Cf-Access-
Authenticated-User-Email' header (verify the Access JWT) for per-user scoping
(HOSTING_PLAN.md:37-38). Local-dev fallback DEV_USER_EMAIL env + an owner_email column + per-
query scoping + narrowed CORS + rate limit + per-user quota are all scoped to the UNSTARTED PR-3
(UI_ALPHA_EXECUTION_PLAN.md:32). Beta graduates to Firebase Auth (HOSTING_PLAN.md:62-63). Net:
ship-blocking gap for any multi-tenant/hosted use; fine only for the single-box local alpha it
currently is.",
    "gaps": [
      "NO 'list videos / list matches' endpoint at all. There is no GET /api/videos and no DB
method to enumerate videos (no get_all_videos/list_videos in database.py; only
get_video(video_id) at database.py:418 which requires you to ALREADY know the id). An
interactive UI cannot populate a match list without first knowing every video_id out-of-band.
UI_PROPOSAL screen A1 'Matches' has no backing endpoint.",
      "NO GET endpoint that returns rallies/segments for a video as JSON. The only way to list
rallies is POST /api/query (backend/main.py:331), which is a body-based RPC, not a RESTful GET
/api/videos/{video_id}/rallies. Functionally usable for per-rally selection (it returns id,
start_time, end_time, duration, metadata, events), but not cacheable/linkable and shaped as a
filter-query, not a resource listing.",
      "NO per-rally thumbnail or frame endpoint. There is no GET
/api/videos/{video_id}/rallies/{id}/thumbnail and no frame-serving route (grep for
/thumbnail|/frame|/proxy = none). A per-rally-selection grid UI cannot show preview stills; no
frame-extraction handler exists.",
      "NO endpoint to serve/stream the SOURCE video or a low-res proxy for scrubbing. Only the
COMPILED reel is streamable (GET /api/highlights/{video_id}/{filename}, backend/main.py:307). A
rally-review UI that lets a user seek/preview each rally in the original footage has nothing to
load (UI_ALPHA_EXECUTION_PLAN M3 'player seek' is unbacked).",
      "Custom-selection -> reel EXISTS (POST /api/compile with arbitrary segment_ids,
backend/main.py:342), but it does NOT return a download URL ? it returns a server-local filepath
('filepath': out_path, backend/main.py:394). The client must separately construct GET
/api/highlights/{video_id}/{output_filename}. A cleaner API would return the ready download
URL.",
      "NO correction / accept-reject / boundary-nudge endpoint. There is no PATCH
/api/segments/{id}; the candidate_segments.human_label/notes columns exist but no route writes
them. This is explicitly the unstarted PR-4 'B6 correction loop'
(UI_ALPHA_EXECUTION_PLAN.md:33). An interactive per-rally UI cannot persist user corrections
today.",
      "NO endpoint links a completed job to a video for follow-up. GET /api/jobs/{job_id}
(backend/main.py:274) returns video_id only while the job row exists; there is no GET
/api/videos/{video_id}/status to re-discover processing state later, and no listing of a video's
compiled reels.",
      "NO /api/health endpoint, despite HOSTING_PLAN.md:42 expecting the SPA to ping one for the
'engine offline' banner. Not built.",
      "Auth/ownership gap for the UI: with no auth and no user scoping, the API cannot show a
per-user match list or prevent one tester from reading/compiling another's rallies
(videos.user_id exists at database.py:221-236 but is never set or filtered on). PR-3 identity
scoping is unstarted (UI_ALPHA_EXECUTION_PLAN.md:32)."
    ],
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\uploads.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\job_worker.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\infrastructure\\database.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\static\\app.js",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\config\\models.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\UI_ALPHA_EXECUTION_PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_PROPOSAL.md"
    ]
  },
  "ui": {
    "spa_plan": {
      "stack": "React + Vite + TypeScript + Tailwind SPA, single-page, client-side router, typed
```

API client; deployed as static hosting (Cloudflare Pages, $0) talking to the existing FastAPI
backend on the owner's GPU box via a Cloudflare Tunnel at api.khelsutra.guru. Auth in alpha =
Cloudflare Access email-allowlist (OTP), zero auth code; backend reads the Cf-Access-
Authenticated-User-Email header for per-user scoping. CLI is NOT replaced ? SPA hits the same
FastAPI surface. Stack 'stands unless owner objects' (still listed open in execution plan). SPA
lives in a SEPARATE private repo `khelsutra` under web/; backend enabler PRs stay in the indexer
repo. Refs: UI_PROPOSAL.md:33-37, UI_ALPHA_EXECUTION_PLAN.md:13-15;35-40,
HOSTING_PLAN.md:20-34.",
      "milestones": [
        "SPA milestones (khelsutra repo web/, UI_ALPHA_EXECUTION_PLAN.md:35-40): M1 scaffold ?
Vite+React+TS+Tailwind, router, typed API client, KhelSutra Guru branding.",
        "M2 ? A2 Upload (chunked + recording-guidelines checklist) + A3 Job Progress (poll).",
        "M3 ? A1 Matches + A4 Rally Review (player seek + accept/reject/nudge wired to backend
PR-4).",
        "M4 ? A5 Highlights (compile + download) + A6 Report + engine-offline banner; plus unit
tests for API client + review-state logic and a clean `npm run build`.",
        "Backend enabler PRs gating the SPA (L1, indexer repo): PR-1 B1 async job model #16
(MERGED #87); PR-2 B2+B3 chunked upload + streaming download (MERGED #99); PR-3 B4+B7 identity &
exposure (owner_email scoping, CORS, rate limit, quota) ? open; PR-4 B6 correction loop (PATCH
/api/segments/{id}) + retire static UI (B5) ? open.",
        "Phase structure (UI_ALPHA_EXECUTION_PLAN.md:19-55): G0 setup ? Phase L (local
build+test: L1 backend, L2 SPA M1-M4, L3 local E2E, GATE GL owner sign-off) ? Phase D
(deployment D1-D3, GATE GD) ? Phase R (rollout wave 1: 5 testers ? widen to 25). Beta is a
separate 4-Drop roadmap (UI_PROPOSAL.md:73-93)."
      ],
      "routes": [
        "A1 Matches (home) ? user's videos, per-video status/verdict chip, upload CTA
(UI_PROPOSAL.md:43)",
        "A2 Upload & Process ? drag-drop, inline recording-guidelines checklist, chunked upload,
auto-start processing (UI_PROPOSAL.md:44)",
        "A3 Job Progress ? stage list (validation?segmentation?AI confirm?report) with live poll
of GET /api/jobs/{id}, verdict banner (UI_PROPOSAL.md:45)",
        "A4 Rally Review (flagged ?, 'the corpus engine') ? player + rally list, click-to-seek,
per-rally Accept/Reject/nudge ±0.5s, bulk 'all good' (UI_PROPOSAL.md:46)",
        "A5 Highlights ? select accepted rallies ? name reel ? compile ? download + list of past
reels (UI_PROPOSAL.md:47)",
        "A6 Report ? customer-audience report (verdict, notes, FAQ refs) + 'nerd view' toggle
for technical.md (UI_PROPOSAL.md:48)"
      ],
      "components": [
        "6-screen alpha set A1-A6 (UI_PROPOSAL.md:39-48).",
        "Rally-review widget: video player + rally list with click-to-seek, accept/reject
toggles, start/end nudge control (±0.5s), bulk-accept, per-rally chips (duration, shot count +
confidence) ? backed by B6 PATCH /api/segments/{id} ? candidate_segments.human_label/notes +
provenance (UI_PROPOSAL.md:46;59, UI_ALPHA_EXECUTION_PLAN.md:33;38).",
        "Chunked uploader (?90-95 MB parts, mp4/mov allowlist, size cap) + recording-guidelines
checklist for pre-upload education (UI_PROPOSAL.md:44;55, HOSTING_PLAN.md:38-39).",
        "Job-progress poller against GET /api/jobs/{id} (UI_PROPOSAL.md:45).",
        "Highlights compiler/download via /api/compile + streaming GET
/api/highlights/{video_id}/{name} (UI_PROPOSAL.md:47;56).",
        "Report renderer with nerd-view toggle (UI_PROPOSAL.md:48).",
        "Typed API client + review-state logic (unit-tested)
(UI_ALPHA_EXECUTION_PLAN.md:36;40).",
        "Engine-offline banner driven by /api/health ping (UI_ALPHA_EXECUTION_PLAN.md:39,
HOSTING_PLAN.md:43)."
      ]
    },
    "conversational_design": "IMPORTANT / honest finding: NONE of the four docs read
(UI_PROPOSAL, UI_ALPHA_EXECUTION_PLAN, HOSTING_PLAN, KHELSUTRA_PRODUCT_OVERVIEW) specify a
conversational 'ask for a reel' query feature, and there is NO mapping of a plain-English
request to TODAY's metadata (length / count / order / which game) anywhere in them. The alpha
Highlights flow (A5) is explicitly a MANUAL select-then-compile: user selects accepted rallies ?
names the reel ? compiles ? downloads (UI_PROPOSAL.md:47). Today's metadata surfaced to the user
is per-rally duration + shot count (+confidence) as chips on the Rally Review screen
(UI_PROPOSAL.md:46) and a summary of rally count + total play time

(KHELSUTRA_PRODUCT_OVERVIEW.md:14;48;65). 'Length and order' appears ONLY as a FEEDBACK prompt to alpha testers ('Whether the reel is the right length and order for how you'd actually use it', KHELSUTRA_PRODUCT_OVERVIEW.md:135) ? i.e. a question asked OF the user, not a query the user can issue. Conversational QA / NL query is EXPLICITLY DEFERRED: 'no conversational QA (deferred in roadmap)' (UI_PROPOSAL.md:66). Where a conversational layer IS specified (outside these four docs, all future/gated): OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55 ('build the bookkeeping NOW, punt the model' ? punt the conversational feature until extraction quality is solid) with the queryable per-video event-index schema as 'the conversational seam' (lines 102, 160); PLATFORM_ARCHITECTURE.md:263-272 (NL queries about rallies/shots/technique, must be localized); I18N_PLAN.md:159 (P4 'Conversational + coach-mode (future): locale-aware NL query intake ? templated localized answers ? gated on those features existing'). So the metadata that COULD back a future query (rally_ordinal, start/end ? length, shot_count, which-game/match keying) is being reserved in the event-index schema (POST_166?MILESTONES.md:55), but no length/count/order/which-game parsing of a plain-English reel request is designed in the read docs. Explicitly deferred dimensions: shot-type and player are both roadmap-only ? shot detection (smashes/drops/net/serves/clears) is 'on the roadmap, not available today' (KHELSUTRA_PRODUCT_OVERVIEW.md:116;160) and player tracking/identification is a hard non-goal / strictly-future biometrics (KHELSUTRA_PRODUCT_OVERVIEW.md:160;181, and the alpha non-goal 'no athlete profiles', UI_PROPOSAL.md:64).",
    "positioning_notes": [
      "Per-rally selection IS in the alpha product, but as TWO distinct surfaces: (a) Rally Review (A4) for accept/reject/boundary-nudge correction, and (b) Highlights (A5) for picking which accepted rallies go into a named reel that is compiled + downloaded. Both are manual UI interactions, not a query/builder (UI_PROPOSAL.md:46-47).",
      "A 'conversational builder' is NOT positioned as a current product feature in any of the four docs. The reel-building experience shipped in alpha is manual select-then-compile (UI_PROPOSAL.md:47); conversational QA is listed as an alpha non-goal, 'deferred in roadmap' (UI_PROPOSAL.md:66).",
      "The strategic framing of per-rally selection is the CORPUS-GROWTH ENGINE, not a consumer convenience: A4 Rally Review is flagged ? as 'the corpus engine' ? every accept/reject/nudge produces the labels the owned-model track is starved for, persisted to candidate_segments.human_label/notes with provenance (UI_PROPOSAL.md:9-14;46;59). This realizes the v1 'correction loop' / 'co-builders' positioning (KHELSUTRA_PRODUCT_OVERVIEW.md:92-94).",
      "A future conversational surface is positioned as a downstream OWNED-MODEL / platform capability, deliberately decoupled: extract-once/query-many event index is the 'conversational seam' to be built early while the conversational *model/feature* is punted until extraction quality is high (OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55;102;160; PLATFORM_ARCHITECTURE.md:263-272). The product overview frames this as the path 'toward an AI assistant coach' ? direction, not today's buttons (KHELSUTRA_PRODUCT_OVERVIEW.md:111-126).",
      "Honesty discipline is explicit in positioning: the overview twice tells users the 'gets sharper over time' / deeper-analysis story is the direction being built toward, NOT a switch already on (KHELSUTRA_PRODUCT_OVERVIEW.md:105-107;124-126)."
    ],
    "available_today": [
      "Upload a full session/match video (phone-on-tripod or GoPro; long videos fine), chunked upload (KHELSUTRA_PRODUCT_OVERVIEW.md:44;47, UI_PROPOSAL.md:44)",
      "Automatic rally detection ? finds rallies + their start/end (KHELSUTRA_PRODUCT_OVERVIEW.md:45)",
      "Highlight reel: rallies stitched into one downloadable/shareable MP4 with a 'Generated by Khelsutra.guru' mark (KHELSUTRA_PRODUCT_OVERVIEW.md:46)",
      "Simple summary: rally count + total actual play time (KHELSUTRA_PRODUCT_OVERVIEW.md:14;48)",
      "Re-use & download: process once, return later, download the reel; re-processing REPLACES the prior result (KHELSUTRA_PRODUCT_OVERVIEW.md:49;66)",
      "Per-rally review + correction: click-to-seek, accept/reject, boundary nudge ±0.5s, bulk-accept, with per-rally duration + shot-count(+confidence) chips (UI_PROPOSAL.md:46) ? the core human correction loop",
      "Manual reel building: select accepted rallies ? name ? compile ? download (UI_PROPOSAL.md:47)",
      "Customer-audience report with verdict/notes/FAQ + nerd-view toggle (UI_PROPOSAL.md:48)",
      "Background/async processing ? close the tab, come back later (KHELSUTRA_PRODUCT_OVERVIEW.md:62-64; async job model PR-1 #87 merged)",
      "Backend reality: alpha serves the Gemini segmenter path (no GPU in request path), ~0.93 F1 on clean single-court footage, ~$0.05/run (UI_PROPOSAL.md:16-18, HOSTING_PLAN.md:32-33)",
      "Invite-only, English-only private web app (KHELSUTRA_PRODUCT_OVERVIEW.md:59;156)"

    ],
    "roadmap": [
      "Shot detection ? smashes, drops, net shots, serves, clears within each rally (KHELSUTRA_PRODUCT_OVERVIEW.md:116;160) ? DEFERRED, not today",
      "Deeper per-rally / per-match analysis: patterns, tendencies (KHELSUTRA_PRODUCT_OVERVIEW.md:117)",
      "Per-match history / progress over weeks-months (KHELSUTRA_PRODUCT_OVERVIEW.md:118)",
      "AI assistant coach ? coaching insights that support, never replace, the coach (KHELSUTRA_PRODUCT_OVERVIEW.md:119-120)",
      "Conversational / NL query (the 'ask for a reel' surface): seam/event-index built now, model/feature punted until extraction quality is solid; must be localized (OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55, PLATFORM_ARCHITECTURE.md:263-272, I18N_PLAN.md:159 P4) ? DEFERRED (UI_PROPOSAL.md:66)",
      "More sports: table tennis next, then other racket/court sports (KHELSUTRA_PRODUCT_OVERVIEW.md:121; alpha is badminton-only, UI_PROPOSAL.md:65)",
      "Localization: Kannada + Hindi alongside English (KHELSUTRA_PRODUCT_OVERVIEW.md:122)",
      "Beta drops (UI_PROPOSAL.md:73-93): Drop 1 accounts/academies + real auth (Firebase/Clerk, 18+ only, roles); Drop 2 cloud storage + GPU box as pull-based byo_worker + direct-to-cloud resumable uploads + email-when-ready; Drop 3 Review v2 (J/K/L + [/] shortcuts, frame-step, timeline minimap, batch accept, tags, expiring share links, per-player aggregates ? NO gait/biometrics); Drop 4 monetization (Razorpay per-match credits) + training-data opt-in (default OFF, adult-only, minors excluded)",
      "Per-person tracking / biometrics: explicitly NOT today and gated as strictly-future (KHELSUTRA_PRODUCT_OVERVIEW.md:107;160;181)",
      "Pricing not set (KHELSUTRA_PRODUCT_OVERVIEW.md:162)"
    ],
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_PROPOSAL.md (alpha 6 screens A1-A6 :39-48; A4 'corpus engine' :46; A5 manual compile :47; conversational QA deferred :66; stack :33-37; beta drops :73-93; backend B1-B7 :52-60)",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_ALPHA_EXECUTION_PLAN.md (SPA milestones M1-M4 :35-40; backend PRs L1 :29-33; phases/gates :19-55; stack-open :15)",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md (alpha topology + Gemini path :20-34; auth header :37-38; 100MB chunk cap :38-39; offline banner :43)",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\KHELSUTRA_PRODUCT_OVERVIEW.md (today vs roadmap :40-49 / :111-126; 'right length and order' feedback prompt :135; shot/player non-goals :160;181; honesty caveats :105-107;124-126)",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\OWNED_MODEL_IMPLEMENTATION_PLAN.md:54-55;102;160 (conversational seam built now, feature punted) ? NOT among the four docs read but the authoritative location of conversational design",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PLATFORM_ARCHITECTURE.md:263-272 (NL query surface, localized) ? supporting",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\I18N_PLAN.md:159 (P4 conversational NL intake, gated) ? supporting",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md:55 (event-index schema fields rally_ordinal, start/end, shot_count) ? the metadata a future query would map onto"
    ]
  },
  "convention": {
    "required_sections": [
      "0. TL;DR ? 2-5 sentences: goal, approach, single most important caveat; call out any standalone-actionable finding (template:30)",
      "1. Problem & goal ? why now; concrete user-facing outcome; what 'good' looks like; link the code/docs a reader must read to get current (template:33). NOTE: real examples often split this into '1. Problem & motivating example' + '2. Goal & non-goals' (PROMINENT_COURT_DETECTION.md:14,19)",
      "2. Decisions locked ? table capturing answered questions so they aren't relitigated (template:35)",
      "3. Foundation ? research / prior art *(OPTIONAL ? include only when the plan rests on external facts: cited research, legal/compliance, benchmarks, prior-art; mark researched-vs-verified; list sign-off gates)* (template:42)",
      "4. Design / architecture ? the actual plan: data contracts, diagrams, and a REUSE MAP

naming existing modules/code this builds on so new-vs-reused is clear (template:45)",
      "5. Threat model / risk table *(OPTIONAL ? security/privacy/compliance work; who/what is protected vs NOT; residual risks stated plainly)* (template:48). In non-security docs this slot is used for a 'Risks & limits' or 'Experiment/measurement plan' section instead (PROMINENT_COURT_DETECTION.md:77,86)",
      "6. Honest limits ? what this does NOT do ? the false-confidence guardrail: what a green suite / DONE status does NOT prove (template:51)",
      "7. Deliverables & progress tracker ? SOURCE OF TRUTH ? one small PR per row, branched from origin/master, no stacking; update row status+PR as work lands (template:54)",
      "8. Open questions ? owner / external ? split blocking-gates from nice-to-knows; name who decides (template:61)",
      "9. Definition of done ? explicit acceptance checklist; when all ticked flip Status->DONE and archive (template:64)",
      "10. References ? Internal code anchors [verified] (paths) · Internal docs · External (cited sources) (template:68)",
      "Changelog (trailing, after a --- rule) ? dated bullets of what changed (template:73)"
    ],
    "header_fields": [
      "Status: a backtick-quoted lifecycle state (DRAFT | IN PROGRESS | DONE) optionally qualified, e.g. `DRAFT` ? `owner-gated for any default flip` or `IN PROGRESS` (template:21, PROMINENT_COURT_DETECTION.md:3, REQUEST_ACCESS_FORM.md:3)",
      "Owner: name, e.g. Avi (template:21, PROMINENT_COURT_DETECTION.md:3)",
      "Created: YYYY-MM-DD (template:21)",
      "Last updated: YYYY-MM-DD (template:21)",
      "Lifecycle: literal `DRAFT ? IN PROGRESS ? DONE ? archived` with the note '(move to docs/archives/ when DONE)' (template:22)",
      "Tracking anchors: '§N progress tracker is the source of truth; indexed in docs/README.md; pointer in memory current-state (and docs/NEXT_STEPS.md once work starts)' (template:23, PROMINENT_COURT_DETECTION.md:5)",
      "Relation to existing docs: <extends | supersedes | peer-of ?> ? which sibling docs/gaps this attacks (template:24, PROMINENT_COURT_DETECTION.md:6)",
      "Honesty note: 'mark each claim [verified] / [researched] / [design]. Not legal/financial advice where applicable.' (template:25, PROMINENT_COURT_DETECTION.md:7)"
    ],
    "progress_tracker_format": "Section §7 titled 'Deliverables & progress tracker' marked '? source of truth' (template:54). It opens with a Legend line then a markdown table. CANONICAL legend (template:55): 'Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. One small PR per row, branched from origin/master, no stacking. Update the row (status + PR link) as work lands.' CANONICAL columns (template:57-59): | ID | Deliverable | Depends on | Gated? | Status | PR | ? first row example: | P0 | ? | ? | No | ? | ? |. IDs are short prefixed codes (P0/P1?, C0/C1?, or tier-coded E1/W1/M1/R1). The Status cell carries the glyph PLUS a short measured result, e.g. '? baseline F1 0.300, merge_rate 0.523' or '? golden ?F1 +0.006 (n.s.)' (RALLY_QUALITY_RESEARCH.md:537-539). Richer variant (RALLY_QUALITY_RESEARCH.md:535) adds a 'Tier' and 'Acceptance / promote criterion' column: | ID | Deliverable | Tier | Depends on | Acceptance / promote criterion | Status |. A measured project also keeps a separate cumulative 'The number' progress table (RALLY_QUALITY_RESEARCH.md:522-531): | after | F1 | merge_rate | ? vs baseline |. Glyph note: template/PROMINENT use ? for done; RALLY_QUALITY_RESEARCH uses ? and its own legend '? not started · ? in progress · ? done' (RALLY_QUALITY_RESEARCH.md:533) ? both accepted.",
    "dod_format": "A dedicated '## 9. Definition of done' section (template:64; PROMINENT_COURT_DETECTION.md:112; REQUEST_ACCESS_FORM.md:101). Format = an explicit acceptance checklist of GitHub-style checkboxes '- [ ] ?', each a verifiable condition (PROMINENT_COURT_DETECTION.md:113-116, REQUEST_ACCESS_FORM.md:102-106). Closing instruction in the template: 'When all ticked ? flip Status ? DONE and archive.' (template:65). For a multi-PR tracked project the DoD may instead be a numbered list of completion clauses tied to tiers/the headline number (RALLY_QUALITY_RESEARCH.md:553-560: 'The project is complete when: 1. ? 2. ? 3. ?'), optionally followed by an owner-adjustable scope note (RALLY_QUALITY_RESEARCH.md:562). Items should be measurable and may carry honesty tags like [measured].",
    "style_notes": [
      "Numbering: sections are numbered §0..§10 starting at '## 0. TL;DR'; §7 is always the progress tracker / source of truth; subsections use dotted numbers (§4.1, §7.1, §7.4) (template:29-69, RALLY_QUALITY_RESEARCH.md:183-560)",
      "OPTIONAL sections (§3 Foundation, §5 Threat model) are included only when relevant and DELETED when they don't apply; their slots are reused for domain-appropriate sections (Risks & limits, Experiment plan) (template:14-15,42,48; PROMINENT_COURT_DETECTION.md:77,86)",

"Status header is a single multi-line blockquote ('> ') at the very top right under the H1 title, fields joined by ' · ' (template:21-25; PROMINENT_COURT_DETECTION.md:3-7; REQUEST_ACCESS_FORM.md:3-9)",
"Honesty discipline is mandatory and pervasive: tag every claim inline with backticked `[verified]` (checked vs code/measured), `[researched]` (needs sign-off), or `[design]` (proposed); never aspirational ? the doc reflects real shipped state (template:11-12,25; PROMINENT_COURT_DETECTION.md:7,14,54). Record refuted/killed claims and corrections explicitly with an 'honesty note' (RALLY_QUALITY_RESEARCH.md:96,364)",
"The doc POINTS to detail (code anchors, research artifacts, sibling docs) and does NOT duplicate it; §7 is the single source of truth (template:10,13; RALLY_QUALITY_RESEARCH.md:17)",
"Code/config references are concrete and clickable: relative markdown links to sibling docs e.g. [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md), and path:line anchors to code e.g. backend/pipeline/detectors/tracknet_runner.py:256 (RALLY_QUALITY_RESEARCH.md:79,581; PROMINENT_COURT_DETECTION.md:57-62)",
"Memory wikilinks [[double-bracket-slug]] cross-reference auto-memory notes, e.g. [[launch-report-convention]], [[weak-signals-retained]], [[decision-rigor-norm]] (PROMINENT_COURT_DETECTION.md:7,12,28)",
"Tone: dense, engineering-precise, plainspoken, opinionated; bold for headline takeaways and the load-bearing word; em-dashes and parentheticals for asides; uses ? for caveats and a 'Dead-ends / do-NOT' list so decisions aren't relitigated (RALLY_QUALITY_RESEARCH.md:261,490; PROMINENT_COURT_DETECTION.md). Every signal/feature change ships flag-gated, default-OFF, promoted only on measured lift",
"Per-row workflow rule baked into the legend: 'One small PR per row, branched from origin/master, no stacking' ? matches CLAUDE.md branching discipline (template:55; PROMINENT_COURT_DETECTION.md:98; REQUEST_ACCESS_FORM.md:79)",
"Trailing '### Changelog' after a horizontal rule (---), dated bullets newest-appended; only present once the doc has live history (template:71-74; PROMINENT_COURT_DETECTION.md:122-126)",
"khelsutra-repo voice (for docs authored there): same status-header + tracker discipline ('mirrors the engine repo's PROJECT_DOC_TEMPLATE'), but tracker may slim to | ID | Deliverable | Gated? | Status | PR |; privacy-first / contact-only-never-a-training-source guardrail echoed; portability framed as 'adapter swap, not a rewrite'; comparison tables with two host/option columns; runbook fenced-code blocks; 'Honest notes' section closing the doc (REQUEST_ACCESS_FORM.md:6,80; PORTABILITY.md:2,15,44)"
],
"file_refs": [
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\RALLY_QUALITY_RESEARCH.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROMINENT_COURT_DETECTION.md",
"C:\\Users\\avidu\\Projects\\khelsutra\\docs\\REQUEST_ACCESS_FORM.md",
"C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PORTABILITY.md"
]
}
}

Return a concrete, buildable design under this lens. api_additions must reference the REAL gaps found. mvp_scope must be the smallest slice that delivers "pick rallies -> get a reel".

2. assistant (2026-06-20T08:03:29.320Z)

I'll ground this design in the actual research provided, then verify a few key facts against the real code before finalizing.

3. user (2026-06-20T08:03:30.726Z)

```
@app.post("/api/query")
def query_play_segments(req: QueryRequest):
    filters = {
        "server": req.server,
        "receiver": req.receiver,
        "duration_min": req.duration_min,
        "event_type": req.event_type
    }
```

```
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}: dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)} segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall below "
                                       f"stitching.min_shot_count={stitching.min_shot_count}.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        # Run compiler with the configured stitching paddings + optional branding watermark
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or "output"
        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding, watermark=stitching.watermark)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
            return {"status": "success", "filepath": out_path}
        except Exception as e:
```

```
        raise HTTPException(status_code=500, detail=f"Highlight stitching failed: {str(e)}") from e

# --- CLI Debug Utility & Orchestration ---
def run_cli_diagnose_clip(video_path: str, timestamp: float):
    """CLI debugging utility to examine segment properties around a timestamp."""
```

4. user (2026-06-20T08:03:31.293Z)

```
    def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
        with self._connect() as conn:
            row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?", (video_id,)).fetchone()
            if row:
                d = dict(row)
                d["validation_results"] = json.loads(d["validation_results"] or "[]")
                return d
        return None

    def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str, Any]]:
        """
        Dynamically queries play segments based on filters:
        - duration_min: float
        - server: string
        - receiver: string
        - event_type: string (e.g. smash, foul)
        """
        query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
        params = [video_id]

        if filters.get("duration_min"):
            query += " AND r.duration >= ?"
            params.append(filters["duration_min"])

        if filters.get("server"):
            query += " AND r.server = ?"
            params.append(filters["server"])

        if filters.get("receiver"):
            query += " AND r.receiver = ?"
            params.append(filters["receiver"])

        if filters.get("event_type"):
            # Check if there is an event of a specific type inside this segment
            query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND e.type = ?)"
            params.append(filters["event_type"])

        segments_list = []
        with self._connect() as conn:
            cursor = conn.cursor()
            rows = cursor.execute(query, params).fetchall()

            for row in rows:
                r_dict = dict(row)
                r_id = r_dict["id"]
                r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                # Fetch associated events
                event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?", (r_id,)).fetchall()
                r_dict["events"] = []
                for er in event_rows:
                    e_dict = dict(er)
```

```
                    e_dict["details"] = json.loads(e_dict["details"] or "{}")
                    r_dict["events"].append(e_dict)

                segments_list.append(r_dict)

        return segments_list

    def clear_video_data(self, video_id: str):
```

5. assistant (2026-06-20T08:03:34.231Z)

Now let me verify the static UI's current state and the candidate_segments correction columns.

6. user (2026-06-20T08:03:35.912Z)

```

            if (!res.ok) throw new Error("Failed to filter index.");

            const data = await res.json();
            renderRallyCards(data.rallies || []);
        } catch (err) {
            alert(`Query failed: ${err.message}`);
        } finally {
            hideLoader();
        }
    });

    // --- 5. COMPILER BOARD MANAGEMENT ---
    const updateCompilerBar = () => {
        const count = selectedRallyIds.size;
        if (count > 0) {
            compileBoard.style.display = "flex";
            compileSelectedCount.textContent = `${count} Rally Segment${count > 1 ? 's' : ''} Selected`;
        } else {
            compileBoard.style.display = "none";
        }
    };

    btnCompile.addEventListener("click", async () => {
        if (!currentVideoId || selectedRallyIds.size === 0) {
            alert("No rallies selected for compiling highlights.");
            return;
        }

        const outputName = highlightOutputName.value.trim() || "highlights_compiled.mp4";
        showLoader("Compiling Highlight Reels", "Triggering localized high-speed FFmpeg stitching demuxers...");

        try {
            const res = await fetch("/api/compile", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({
                    video_id: currentVideoId,
                    // API field is segment_ids (the old rally_ids never matched the request model)
                    segment_ids: Array.from(selectedRallyIds),
                    output_filename: outputName
                })
            });

            if (!res.ok) {
                const err = await res.json();
```

```
                    throw new Error(err.detail || "Concatenation failed.");
                }

                await res.json();
                // B3 (PR-2): offer the reel via the streaming download endpoint instead of
                // alerting a server-local path the browser can't reach.
                const dlUrl = `/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;
                let dl = document.getElementById("highlight-download-link");
                if (!dl) {
                    dl = document.createElement("a");
                    dl.id = "highlight-download-link";
                    dl.style.cssText = "display:block;margin-top:10px;font-weight:bold;";
                    compileBoard.appendChild(dl);
                }
                dl.href = dlUrl;
                dl.setAttribute("download", outputName);
                dl.textContent = `? Download ${outputName}`;
                alert(`Highlight compilation completed! Use the download link to save ${outputName}.`);

            } catch (error) {
                alert(`Stitching compilation failed: ${error.message}`);
            } finally {
                hideLoader();
            }
        });
    });

```

7. user (2026-06-20T08:03:36.610Z)

```
        def _migrate_to_v1(self, cursor):
            # v1: raw candidate detections (pre-confirmation), detector-agnostic.
            # Common fields are columns; detector-specific metrics go in `stats` JSON,
            # so a new segmenter reuses this table by setting its own `detector`/`stats`.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS candidate_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    detector TEXT NOT NULL,
                    chunk_index INTEGER,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    confidence REAL,
                    stats TEXT,
                    detection_params TEXT,
                    confirmed_segment_id INTEGER,
                    human_label TEXT,
                    notes TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                    FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON DELETE SET NULL
                )
            """)
            cursor.execute("""
                CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                    ON candidate_segments(video_id, detector)
            """)
            cursor.execute("PRAGMA user_version = 1")

        def _migrate_to_v2(self, cursor):
            # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
```

```
        # /api/process request. `status` is the EXECUTION state of the job
        # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
        # that failed validation) lives inside `result` JSON as result["status"].
        # `result` is the full sync-era response dict (incl. report paths), so the
        # observability report is the job's artifact, per the plan.
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS jobs (
                id TEXT PRIMARY KEY,
                video_path TEXT NOT NULL,
                video_id TEXT,
                segmenter TEXT,
                player_pool TEXT,
                max_frames INTEGER,
                status TEXT NOT NULL DEFAULT 'queued',
                progress TEXT,
                error TEXT,
                result TEXT,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                started_at TIMESTAMP,
                finished_at TIMESTAMP
            )
        """)
        cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
        cursor.execute("PRAGMA user_version = 2")

    def _migrate_to_v3(self, cursor):
        # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
        # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is due
        # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
        # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
        # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
        # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run them.
        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
        cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status, next_poll_at)")
        cursor.execute("PRAGMA user_version = 3")

    def _migrate_to_v4(self, cursor):
```

8. assistant (2026-06-20T08:03:42.251Z)

I have enough verified ground truth. The research is accurate against the code: `/api/query` returns `{play_segments: [...]}` (not `rallies`, confirming the app.js bug at line 344), `/api/compile` takes `segment_ids` and returns a server-local `filepath`, `query_play_segments` is the only list path, there's no source/proxy/thumbnail streaming, and the correction columns (`human_label`, `notes`, `confirmed_segment_id`) exist but nothing writes them. Now I'll produce the design.

9. user (2026-06-20T08:04:59.781Z)

Structured output provided successfully

10. assistant (2026-06-20T08:05:12.923Z)

I've delivered the design via the StructuredOutput tool, grounded in the verified engine reality.

Key things I confirmed against the actual code before finalizing:

- **The read path and its bug**: `POST /api/query` returns `{play_segments:[...]}`

(database.py:475), but `app.js:344` reads `data.rallies` ? the known bug. Fixing this is a prerequisite for the MVP and the only list path that exists.
- **The compile contract**: `POST /api/compile` (main.py:342-396) takes `{video_id, segment_ids, output_filename}`, silently applies `stitching.min_shot_count` (exempting unknown counts), and returns a server-local `filepath` ? NOT a download URL. The client already reconstructs `GET /api/highlights/{video_id}/{filename}` by hand (app.js:392).
- **What's actually selectable per rally**: only `start_time`, `end_time`, `duration`, and `metadata.shot_count(+confidence)` on the Gemini path. `server`/`receiver`/events are fabricated motion-only values; shot_type/player/score/game-set don't exist anywhere.

The core design decision driven by the timeline-first lens and the real gaps: the MVP "pick rallies ? get a reel" loop needs **zero new backend endpoints** (it rides query + compile + highlights), but the "timeline-first" promise of inline preview/scrub is **blocked** by the absence of any source-video stream ? the single highest-value, ship-blocking `api_addition` (`GET /api/videos/{video_id}/source`). I kept that honest rather than pretending the dominant surface is fully buildable today.

Per the repo convention, a buildable version of this should land as a tracked project doc (`PROJECT_DOC_TEMPLATE.md` with a §7 progress tracker) with any engine-side enabler flagged default-OFF before code ? but I did not write any `.md` file, per the instructions.

Files I read/grounded against (all absolute):
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py` (331-396)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py` (150-217, 418-475)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\static\app.js` (340-411)